

Python Fundamentos

Entornos virtuales

1



Las aplicaciones desarrolladas en Python a menudo requieren el uso de paquetes y módulos que no están incluidos en la librería estándar. Además, estas aplicaciones a veces necesitan versiones específicas de determinados paquetes.

Esto significa que puede resultar difícil para una instalación única de Python satisfacer los requisitos de todas las aplicaciones. Por ejemplo, si la aplicación A necesita la versión 1.0 de un módulo y la aplicación B necesita la versión 2.0, surge un conflicto en los requisitos. Instalar la versión 1.0 o 2.0 podría dejar una de las aplicaciones sin funcionar correctamente.

La solución a este dilema es la creación de un entorno virtual. Un entorno virtual es un directorio que contiene una instalación específica de Python, junto con los paquetes adicionales necesarios. De esta manera, diferentes aplicaciones pueden utilizar entornos virtuales independientes para garantizar que cumplan con sus requisitos específicos sin interferencias entre ellas.



Un entorno virtual es una herramienta que proporciona a los desarrolladores una forma de aislar las dependencias y paquetes de sus proyectos Python. Esto es especialmente útil cuando trabajas en múltiples proyectos que requieren diferentes versiones de paquetes o bibliotecas.

Características:

- **Aislamiento de dependencias:** Cada proyecto puede tener sus propias dependencias, lo que evita conflictos entre paquetes de diferentes proyectos
- **Reproducibilidad:** Permite recrear el entorno de desarrollo exacto en otros sistemas, asegurando que el proyecto funcione de la misma manera en diferentes máquinas
- **Manejo de versiones:** Facilita el trabajo con diferentes versiones de Python y paquetes sin interferencias

venv

2



Venv es el módulo estándar de Python para crear y manejar entornos virtuales. Permite crear entornos de desarrollo aislados de manera sencilla y efectiva.

Lo primero que tenemos que hacer es instalarlo ejecutando desde cmd **pip install virtualenv**

```
>> prueba_venv 19:09 pip install virtualenv
Collecting virtualenv
  Downloading virtualenv-20.26.3-py3-none-any.whl.metadata (4.5 kB)
Collecting distlib<1,>=0.3.7 (from virtualenv)
  Downloading distlib-0.3.8-py2.py3-none-any.whl.metadata (5.1 kB)
Collecting filelock<4,>=3.12.2 (from virtualenv)
  Downloading filelock-3.15.4-py3-none-any.whl.metadata (2.9 kB)
Collecting platformdirs<5,>=3.9.1 (from virtualenv)
  Downloading platformdirs-4.2.2-py3-none-any.whl.metadata (11 kB)
Downloading virtualenv-20.26.3-py3-none-any.whl (5.7 MB)
 5.7/5.7 MB 5.7 MB/s eta 0:00:00
Downloading distlib-0.3.8-py2.py3-none-any.whl (468 kB)
 468.9/468.9 kB 9.8 MB/s eta 0:00:00
Downloading filelock-3.15.4-py3-none-any.whl (16 kB)
Downloading platformdirs-4.2.2-py3-none-any.whl (18 kB)
Installing collected packages: distlib, platformdirs, filelock, virtualenv
Successfully installed distlib-0.3.8 filelock-3.15.4 platformdirs-4.2.2 virtualenv-20.26.3

[notice] A new release of pip is available: 24.0 -> 24.2
[notice] To update, run: python.exe -m pip install --upgrade pip
>> prueba_venv 19:11 |
```




Cuando instalamos paquetes en Python sin un entorno virtual estos paquetes se instalan de forma global. Esto significa que se instalan directamente en el entorno de Python del sistema y que una vez instalados, estos paquetes están disponibles para cualquier script o proyecto Python en esa máquina.

Podemos ver que paquetes tenemos instalados de forma global, ejecutando desde cmd el comando **pip freeze**

```
>> prueba_venv 19:25 pip freeze
distlib==0.3.8
filelock==3.15.4
platformdirs==4.2.2
virtualenv==20.26.3
```

Ahora si instalamos otro paquete de forma global podemos ver como ya nos aparece. Instalamos el paquete 'pandas' para este ejemplo

```
>> prueba_venv 19:25 pip install pandas
Collecting pandas
  Downloading pandas-2.2.2-cp312-cp312-win_amd64.whl.metadata (19 kB)
Collecting numpy>=1.26.0 (from pandas)
  Downloading numpy-2.0.1-cp312-cp312-win_amd64.whl.metadata (60 kB)
    60.9/60.9 kB 1.1 MB/s eta 0:00:00
Collecting python-dateutil>=2.8.2 (from pandas)
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting pytz>=2020.1 (from pandas)
  Downloading pytz-2024.1-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata>=2022.7 (from pandas)
  Downloading tzdata-2024.1-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas)
  Downloading six-1.16.0-py2.py3-none-any.whl.metadata (1.8 kB)
  Downloading pandas-2.2.2-cp312-cp312-win_amd64.whl (11.5 MB)
    11.5/11.5 MB 7.2 MB/s eta 0:00:00
  Downloading numpy-2.0.1-cp312-cp312-win_amd64.whl (16.3 MB)
```

```
>> prueba_venv 19:58 pip freeze
distlib==0.3.8
filelock==3.15.4
numpy==2.0.1
pandas==2.2.2
platformdirs==4.2.2
python-dateutil==2.9.0.post0
pytz==2024.1
six==1.16.0
tzdata==2024.1
virtualenv==20.26.3
```



Ahora vamos a ver como crear, activar, instalar y eliminar paquetes en el entorno virtual.

Primero tenemos que crearnos el entorno virtual. Para esto, desde cmd en la ruta del proyecto, ejecutamos **virtualenv nombre_entorno**

```
>> prueba_venv 20:27 virtualenv prueba_venv
created virtual environment CPython3.12.3.final.0-64 in 6775ms
  creator CPython3Windows(dest=C:\Users\juan.navarrete\Desktop\proyectos\pyth
o_vcs_ignore=False, global=False)
  seeder FromAppData(download=False, pip=bundle, via=copy, app_data_dir=C:\Us
  added seed packages: pip==24.1
  activators BashActivator,BatchActivator,FishActivator,NushellActivator,Power
>> prueba_venv 20:28 |
```

Para este ejemplo le llamo como al proyecto pero puede ser cualquier nombre. Si nos vamos a la carpeta del proyecto podremos ver como se ha creado otra con el nombre del entorno virtual que acabamos de crear.



El siguiente paso es activar el entorno virtual para poder instalar paquetes. Para esto, desde cmd en la carpeta del proyecto, ejecutamos **.\nombre_entorno\Scripts\activate**.

Vemos como se ha activado el entorno virtual y ahora si ejecutamos **pip freeze** no nos aparece ningún resultado, porque no tenemos ningún paquete instalado. Con esto ya podemos empezar a instalar paquetes como lo hacíamos de forma global y se instalarán en el entorno virtual que acabamos de crear y activar.

```
>> prueba_venv 20:37 .\prueba_venv\Scripts\activate
(prueba_venv) >> prueba_venv 20:37 |
```

```
>> prueba_venv 20:37 .\prueba_venv\Scripts\activate
(prueba_venv) >> prueba_venv 20:37 pip freeze
(prueba_venv) >> prueba_venv 20:39 |
```

Volvemos a instalar pandas y comprobamos como se ha instalado en el entorno virtual

```
(prueba_venv) >> prueba_venv 20:43 pip freeze
numpy==2.0.1
pandas==2.2.2
python-dateutil==2.9.0.post0
pytz==2024.1
six==1.16.0
tzdata==2024.1
(prueba_venv) >> prueba_venv 20:44 |
```



Ahora vamos a eliminar el paquete que hemos instalado en el paso anterior, ejecutando desde cmd el comando **pip uninstall nombre_paquete**. También podemos desinstalar varios a la vez poniendo un paquete tras otro separados por un espacio.

Por último desactivamos el entorno virtual ejecutando desde cmd el comando **deactivate**

```
(prueba_venv) >> prueba_venv 20:52 deactivate  
>> prueba_venv 20:52 |
```

Con esto ya tenemos todos los pasos para trabajar con el entorno virtual venv en nuestros proyectos

Instalación librerías

3



Como se mencionó anteriormente, Python cuenta con una amplia variedad de bibliotecas para realizar diversas tareas. A continuación, aprenderemos a instalar bibliotecas externas. Para instalar una biblioteca, simplemente debes ingresar al entorno virtual y escribir el siguiente comando:

```
bash
```

```
pip install {biblioteca}
```

Al ejecutar este comando, observarás que comienza la descarga e instalación de los paquetes necesarios para la biblioteca en cuestión. Este proceso automatizado a través de `pip` facilita la incorporación de nuevas funcionalidades a tus proyectos, permitiéndote acceder a la amplia gama de bibliotecas disponibles en el ecosistema de Python.

```
Collecting pandas  
  Downloading pandas-1.2.0-cp38-cp38-win_amd64.whl (9.3 MB)  
    | 5.2 MB 930 kB/s eta 0:00:05
```



Has señalado una práctica eficiente en el desarrollo de proyectos en Python. En efecto, cuando te unes a un proyecto existente, especialmente aquellos de mayor envergadura, es común encontrarse con un archivo llamado `requirements.txt`. Este archivo contiene una lista de todas las dependencias necesarias para que el código se ejecute correctamente.

Para instalar todas estas dependencias de manera sencilla y eficiente, solo necesitas ejecutar el siguiente comando:

```
bash
```

```
pip install -r requirements.txt
```

La opción `-r` se utiliza para indicar que las dependencias deben instalarse de manera recursiva, lo que significa que también se instalarán las dependencias de las dependencias, y así sucesivamente. Esta práctica simplifica significativamente el proceso de configuración de un entorno de desarrollo, ya que se encarga automáticamente de instalar todas las bibliotecas necesarias según las especificaciones del archivo `requirements.txt`.

```
requirements: Bloc de notas
Archivo  Edición  Formato  Ver  Ayuda
argon2-cffi==20.1.0
async-generator==1.10
attrs==20.3.0
backcall==0.2.0
bleach==3.2.1
cffi==1.14.4
colorama==0.4.4
decorator==4.4.2
defusedxml==0.6.0
entrypoints==0.3
ipykernel==5.4.2
ipython==7.19.0
ipython-genutils==0.2.0
ipywidgets==7.5.1
jedi==0.17.2
Jinja2==2.11.2
jsonschema==3.2.0
jupyter==1.0.0
jupyter-client==6.1.7
jupyter-console==6.2.0
jupyter-core==4.7.0
jupyterlab-pygments==0.1.2
MarkupSafe==1.1.1
mistune==0.8.4
```

¡Gracias!